Лекция 7.
Обобщённая типовая система, лямбда-куб
Интуиционистская типовая система

Терминология: типы, рода, сорта

Тип - сорт
род - сорт
сорт - сорт

Мы будем рассматривать конструкции следующих сортов:

Название сорта	Примеры сортов	Примеры конструкций, имеющих сорт
Тип	α, $\alpha \to \beta$, $\star \to \alpha$	$3 : \texttt{int}, \texttt{id} : \forall\alpha.\alpha \to \alpha$
Род (kind)	$\star$, $\star \to \star$, $\alpha \to \star$	$\texttt{list} : \star \to \star$
Сорт (1)	$\square$	$\star \to \star : \square$

Сорт.

Язык обобщённой типовой системы

Откажемся от различных пространств имён для значений, типов и прочего, а также от синтаксического их разделения. Все переменные для значений любого сорта, все лямбда-выражения для любых функций — всё записывается единообразно.

Определение (синтаксис выражений)

Константы сортов: $c ::= \{\star, \Box\}$

Выражение:

a вып. a. тип.

$$T ::= x \mid c \mid T\ T \mid \lambda x^T.\ T \mid \Pi x^T.\ T$$

$\mid [T] \ldots$

Сокращения:

$$A \to B ::= \Pi x^A.B, \quad x \notin FV(B)$$

$$\forall x.P ::= \Pi x^\star.P \quad \Lambda x.\sigma ::= \lambda x^\star.\sigma$$

$\Lambda x.\sigma : \forall x.P$

$\lambda x^*.\sigma : \Pi x^*.P$

Метапеременные термов: $A, B, C, \ldots, \quad \rho, \sigma, \tau, \ldots.$

Метапеременные переменных: x, y, z

Что такое П

Неформально: П — аналог лямбда-выражения для типизации конструкции:

$$\lambda x^{\tau}.P : \Pi x^{\tau}.\pi$$

Вспомним сокращения:

$$A \to B ::= \Pi x^{A}.B, \quad x \notin FV(B) \qquad \forall x.P ::= \Pi x^{\star}.P$$

И рассмотрим map:

$$\texttt{map} : \forall a.\forall b.(a \to b) \to [a] \to [b]$$

Н.М. $(a \to b) \to [a] \to [b]$

Перепишем:

$$\texttt{map} : \Pi a^{\star}.\Pi b^{\star}.(\Pi f^{\Pi x^{a}.b}.\Pi l^{[a]}.[b])$$

$b \to [b]$

Заметим, что операция $[\sigma]$ строит из σ другой тип, то есть $[\sigma] = (\lambda x^{\star}.\langle \text{тип реализации списка из } x\rangle)\sigma$, можем раскрыть дальше:

$$\texttt{map} : \Pi a^{\star}.\Pi b^{\star}.(\Pi f^{\Pi x^{a}.b}.\Pi l^{(\lambda x^{\star}\dots)a}.(\lambda x^{\star}\dots)b)$$

Заметим, что $\lambda \sigma^{\star}.[\sigma] : \star \to \star$

$\Pi \langle^{\star}.P\langle 3\rangle$

Обобщённая типовая система: семейство систем

Семейство параметризовано множеством пар $\mathcal{S} \subseteq \{\star, \Box\} \times \{\star, \Box\}$

$C : *$

$C : \Box$

Аксиома:

$$\frac{}{\vdash \star : \Box}$$

Общие правила вывода: $\sigma \in \{\star, \Box\}$

а имеет тип.

C — либо тип, либо род.

(Акс.)
$$\frac{\Gamma \vdash A : \sigma}{\Gamma, x : A \vdash x : A}\ x \notin \Gamma \qquad \frac{\Gamma \vdash A : B \qquad \Gamma \vdash C : \sigma}{\Gamma, x : C \vdash A : B}$$
(осл.)

(Втор.)
$$\frac{\Gamma \vdash A : B \qquad \Gamma \vdash B' : \sigma \qquad B =_\beta B'}{\Gamma \vdash A : B'} \qquad \frac{\Gamma \vdash F : (\Pi x^A.B) \qquad \Gamma \vdash H : A}{\Gamma \vdash (F\ H) : B[x := H]}$$
(удал. П)

Пит*.

$*:\Box$

$\frac{\alpha : \sigma}{\alpha : \sigma}$

Частные правила: $\langle \sigma_1, \sigma_2 \rangle \in \mathcal{S}$

Г: $\forall x.P. = \Pi x^*.P$

анал. удал →

отр. возм. подтр.

(ИМ пр.)
$$\frac{\Gamma \vdash A : \sigma_1 \qquad \Gamma, x : A \vdash B : \sigma_2}{\Gamma \vdash (\Pi x^A.B) : \sigma_2}\ \text{П-правило}$$

(подтр. →П)

$$\frac{\Gamma \vdash A : \sigma_1 \qquad \Gamma, x : A \vdash P : B \qquad \Gamma, x : A \vdash B : \sigma_2}{\Gamma \vdash (\lambda x^A.P) : (\Pi x^A.B)}\ \lambda\text{-правило}$$

(введ. →П)

Типизация $\Lambda\alpha.\lambda x^{\alpha}.x$

$\Lambda\alpha.\lambda x^{\alpha}.x : \Pi\alpha^{*}.\alpha \to \alpha$ $\quad$ A $\quad$ $\Pi x^{\alpha}.\alpha$

template<typename T> T id(T x) {return x;}

Выражение $\Lambda\alpha.\lambda x^{\alpha}.x$ перепишется как $\lambda\alpha^{\star}.\lambda x^{\alpha}.x$, ожидаем тип $\Pi\alpha^{\star}.\Pi x^{\alpha}.\alpha$.

Потребуются частные правила для $\langle\star,\star\rangle$ и $\langle\Box,\star\rangle$.

ф. $\quad$ полим. ф. (генерики)

$$\frac{\Gamma \vdash A : \sigma_1 \qquad \Gamma, x : A \vdash B : \sigma_2}{\Gamma \vdash (\Pi x^{A}.B) : \sigma_2} \qquad \frac{\Gamma \vdash A : \sigma_1 \qquad \Gamma, x : A \vdash P : B \qquad \Gamma, x : A \vdash B : \sigma_2}{\Gamma \vdash (\lambda x^{A}.P) : (\Pi x^{A}.B)}$$

$\vdash * : \Box$ $\quad$ a id(a x) {return x;} $\quad$ ξ (продукция)

$$\dfrac{\dfrac{}{\vdash \star : \Box}\;A \qquad \dfrac{\dfrac{}{a : \star \vdash a : \star}\;A \qquad \dfrac{}{a : \star, x : a \vdash x : a} \qquad \dfrac{}{a : \star, x : a \vdash a : \star}}{a : \star \vdash \lambda x^{a}.x : \Pi x^{a}.a}\;\langle\star,\star\rangle \qquad \dfrac{a : \star, x : a \vdash a : \star}{a : \star \vdash \Pi x^{a}.a : \star}\;\langle\Box,\star\rangle}{\vdash \lambda a^{\star}.\lambda x^{a}.x : \Pi a^{\star}.\Pi x^{a}.a}$$

A $\quad$ A $\quad$ B $\quad$ B $\quad$ $\alpha \to \alpha : *$

$a \to a \equiv \Pi x^{a}.a$, если x исп.

Общие свойства обобщённой системы типов

Теорема

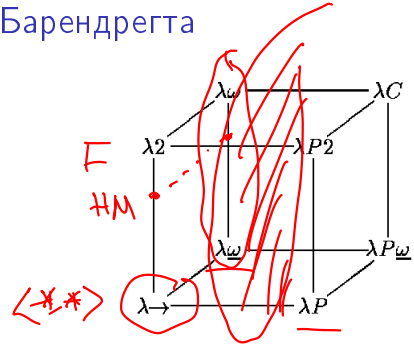
Для обобщённой системы типов выполнена теорема Чёрча-Россера

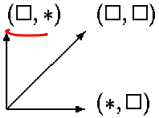
Теорема

Обобщённая система типов сильно нормализуема

Мерауд. в Баз. ст.

Лямбда-куб Барендрегта





$S = (*, \square)^2$

⟨*,*⟩ есть всегда

⟨*, □⟩

⟨□, *⟩

⟨□, □⟩

Типовые системы и языки программирования:

Классические и функциональные языки:

эк.	$\lambda_{\rightarrow}$	$\{\langle\star,\star\rangle\}$	Классический Паскаль
тип→зн.	$\lambda_{\underline{\omega}}$	$\{\langle\star,\star\rangle,\langle\square,\star\rangle\}$	Система F
тип→тип тип→зн.	λ_{ω}	$\{\langle\star,\star\rangle,\langle\square,\star\rangle,\langle\square,\square\rangle\}$	Haskell, Ocaml

Языки с зависимыми типами данных (обычно около λC):

Idris, Coq, Agda, Arend, C++ :).

list: ~~∗ → ∗~~

$\Pi\alpha^{*}.[\alpha]:\square$

Изоморфизм Карри-Ховарда

$\Pi x^{*}.\pi$

Рассмотрим формулу с квантором: $\forall x.\pi$. Ей соответствует $\Pi x.\pi$, а доказательство было бы $\lambda x.P : \Pi x.\pi$. Подробнее:

$\lambda x^{\star}.P : \Pi x^{\star}.\pi : \star$ $\quad$ $x \in V$, для логики 2 порядка

$\lambda x^{\upsilon}.P : \Pi x^{\upsilon}.\pi : \star$, если $\upsilon : \star$ $\quad$ $x \in U \subseteq D$, для (многосортной) логики 1 порядка

В самом деле: $\forall x.\pi$ требует $\pi[x := \theta]$ при всех θ (соответствующих υ).
Доказательство: функция $\lambda x.P$, отображающая θ в терм, обитающий в $\Pi x.\pi$.

x: уел.

Логика	λ-исчисление	Комментарий
π	$x : \pi$	Утверждение
$\pi(x)$	$P : \pi(x)$	Предикат
$\forall x \in U.\pi$	$\lambda x^{\upsilon}.P : \Pi x^{\upsilon}.\pi$	Тотальная функция
$\exists x \in U.\varepsilon$	$(X, U[x := X]) : \Sigma x^{\upsilon}.\varepsilon$	Зависимая пара

: int[x]

[7] : int[1]

[7, 12] : int[2]

Idris: пример языка с зависимыми типами

```
data Nat : Type where
   Z : Nat
   S : Nat -> Nat
```

конст.

ф-я 1 арг.

S (S (S Z))

```
data Vect : Nat -> Type -> Type where
   Nil  : Vect Z a
   (::) : a -> Vect k a -> Vect (S k) a

(++) : Vect n a -> Vect m a -> Vect (n + m) a
(++) Nil       ys = ys
(++) (x :: xs) ys = x :: xs ++ ys
```

hd: Vect (S x) a.

hd (Nil)

Зависимые типы: printf на Идрис

```
-- Mukesh Tiwari, https://github.com/mukeshtiwari/Idris/blob/master/Printf.idr
data Format = FInt Format
            | FString Format
            | FOther Char Format
            | FEnd
```

'%d ... %s ...'

```
format : List Char -> Format
format ('%' :: 'd' :: cs ) = FInt ( format cs )
format ('%' :: 's' :: cs ) = FString ( format cs )
format ( c :: cs )         = FOther c ( format cs )
format []                  = FEnd
```

"%d%s":
Int -> String -> Str.

```
interpFormat : Format -> Type
interpFormat ( FInt f )      = Int -> interpFormat f
interpFormat ( FString f )   = String -> interpFormat f
interpFormat ( FOther _ f )  = interpFormat f
interpFormat FEnd            = String
```

interpFormat:
$\lambda x : \overset{\text{String}}{\ } . \text{Type} : \square$
$\langle *, \square \rangle$

Printf на Идрис

```
formatString : String -> Format
formatString s = format ( unpack s )

toFunction : ( fmt : Format ) -> String -> interpFormat fmt
toFunction ( FInt f ) a     = \i => toFunction f ( a ++ show i )
toFunction ( FString f ) a  = \s => toFunction f ( a ++ s )
toFunction ( FOther c f ) a = toFunction f ( a ++ singleton c )
toFunction FEnd a           = a

sprintf : ( s : String ) -> interpFormat ( formatString s )
sprintf s = toFunction ( formatString s ) ""

main : IO ()
main = putStrLn (sprintf "String: %s, integer: %d" "alpha" (10+23))
```

Интуиционистская теория типов

- Пер Мартин-Лёф, 1972 год (An Intuitionistic Theory of Types), 1982 год (Constructive mathematics and computer programming).
- Примерно соответствует расширенному варианту λ_C из лямбда-куба (зависимые типы, Π и Σ типы и т.п.).
- Явно введённые в теорию индуктивные типы (расширение алгебраических типов) и W-типы.
- Иерархия универсумов.
- Специальный тип для равенства. $=$

$239 \overset{?}{\sim} \overset{17}{234}$

Равенство

- Структурное равенство: побитовая одинаковость. Разрешимо, но малопригодно для жизни:

$$\frac{2}{5} \neq \frac{4}{10}$$

- Доказательное равенство: на основании аксиоматики. Неразрешимо.

Равенство в ИТТ

- Введём отношение идентичности $I(A, a, b)$:

$$\frac{\Gamma \vdash A\ \mathbf{type} \qquad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash I(A, a, b)\ \mathbf{type}}$$

- Теория 1972 года с интенсиональным равенством:

$$\frac{\Gamma \vdash A\ \mathbf{type} \qquad \Gamma \vdash a : A}{\Gamma \vdash \mathbf{refl}_A(a) : I(A, a, a)}$$

- Теория 1982 года — с доказательным равенством. Вводим отношение равенства $(=)$, для него принимаются аксиомы и правила вывода для равенства, и вводим «каноническое доказательство» $\mathbf{r}$:

$$\frac{\Gamma \vdash a =_A b}{\Gamma \vdash \mathbf{r} : I(A, a, b)} \qquad \frac{\Gamma \vdash c : I(A, a, b)}{\Gamma \vdash a =_A b} \qquad \frac{\Gamma \vdash c : I(A, a, b)}{\Gamma \vdash c = \mathbf{r} : I(A, a, b)}$$

Наличие канонического доказательства позволяет выводить равенство доказательств идентичности.

Гомотопическая Т.Т. Изоморфизм Карри-Ховарда-Воеводского

Владимир Александрович Воеводский, 2006 год (A very short note on homotopy λ-calculus).

Логика	λ-исчисление	Топология
Высказывание	Тип	Пространство
Доказательство	Терм	Точка в пространстве
Предикат	Зависимый тип	Расслоение
Равенство	*Выделенный* зависимый тип	Пространство путей
Доказательство равенства	Элемент равенства	Путь между точками

Определение

Будем считать два значения a и b в пространстве X равными, если они связаны путём: непрерывной функцией $f : I \to X$, где $I = [0, 1]$, такой, что $f(0) = a$, $f(1) = b$.

Из определения равенства по Лейбницу следует, что если некоторый предикат истинен для левой части непрерывного пути, то он истинен и для правой части.

Равенство на примере натуральных чисел

Определение

$a, b \in \mathbb{N}_0$. Тогда $a \equiv b$, если существует непрерывная $f : I \to \mathbb{N}_0$, причём $f(0) = a$, $f(1) = b$

Лемма

$a \equiv b$ тогда и только тогда, когда $a = b$.

Доказательство.

Пусть $a = b$. Положим $f(x) = a$. Тогда $f(0) = a = b = f(1)$.
Пусть $a \neq b$. Предположим, есть путь f. Рассмотрим $A = f^{-1}(a)$ и $X = f^{-1}(\mathbb{N}_0 \backslash \{a\})$. По дискретности топологии $\{a\}$ и $\mathbb{N}_0 \backslash \{a\}$ открыты. По непрерывности тогда A и X открыты. Также оба непусты (очевидно, $0 \in A$, $1 \in X$), $A \cap X = \varnothing$ и $A \cup X = I$. Значит, I несвязно, что не так. Противоречие. $\square$

О языке Аренд

- Разработан в Jet Brains.
- Назван в честь Аренда Гейтинга (Arend Heyting, 9 мая 1898, Амстердам — 9 июля 1980, Лугано) — буква «H» из BHK-интерпретации.
- Язык сейчас исключительно для доказательств математических фактов: в отличие, например, от Coq — широко применяется для верификации софта, может использоваться для построения кода по доказательствам.
- Основан на Гомотопической теории типов (HoTT) — развитии интуиционистской теории типов Мартина-Лёфа.

Краткий перечень языковых конструкций

- Синтаксис: отчасти вдохновлён TeX-ом.
- Реализует конструкции из λC: зависимые типы, пи- и сигма-типы.

```
\func fe : \Pi (P : Nat -> \Type) ->
          (\Pi (a : Nat) -> (P a)) -> \Sigma (x : Nat) (P x) =>
   \lam P proof_forall => (0, proof_forall 0)
```

  «если утверждение P истинно при всех натуральных аргументах, то существует натуральный аргумент, при котором оно истинно»
- Индуктивные типы — обобщение алгебраических.
- Предикативная иерархия универсумов (базовые типы имеют уровень 0, тип уровня k имеет уровень $k + 1$).
- Гомотопическая теория типов — формализуется с помощью унивалентной кубической теории типов. Равенство понимается как путь — в отличии от системы индуктивных типов в теории типов Мартина-Лёфа.

В данных слайдах мы представим по несколько примеров для каждого из пунктов (кроме иерархии универсумов), отсылая читателя/слушателя к документации для разбора более тонких вопросов.

Индуктивные типы

- Алгебраические типы: параметры типа не влияют на выбор конструктора

```
type 'a list = Nil | Cons a ('a list)
```

- Обобщённые алгебраические типы: допустимые конструкторы зависят от параметров типа

```
type _ constant =
  | Int : int -> int constant
  | Float : float -> float constant

Int 5 : int constant
Float 3.141 : float constant
```

- Обобщаем дальше (зависимые типы в параметрах, сложные конструкторы): W-типы, индуктивные типы в разных вариантах.

Индуктивные типы в Аренде

Простой случай — похоже на алгебраические типы:

```
\data Nat
   | zero
   | suc Nat
```

Случай сложнее:

```
\data Fin (x : Nat) \elim x
  | suc n => { fzero | fsuc (Fin n) }

fzero : Fin 15          -- ок
fsuc (fzero) : Fin 1    -- не типизируется
```

Аренд интересен наличием высших индуктивных типов, простой пример:

```
\data Int
  | \coerce pos Nat
  | neg Nat \with { zero => pos zero }
```

Элиминаторы для индуктивных типов

- Знакомы с элиминаторами начиная с алгебраических типов:

$$\frac{\Gamma \vdash L : \varphi \vee \psi, \quad \Gamma \vdash f : \varphi \to \tau, \quad \Gamma \vdash g : \psi \to \tau}{\Gamma \vdash \mathsf{Case}\ L\ f\ g : \tau}$$

- Вспомним натуральные числа:

```
\data Nat
    | zero
    | suc Nat
```

Построим зависимый элиминатор:

```
\func Nat-elim (P : Nat -> \Type)
               (z : P zero)
               (s : \Pi (n : Nat) -> P n -> P (suc n))
               (x : Nat) : P x \elim x
  | zero => z
  | suc n => s n (Nat-elim P z s n)
```

Доказательство с помощью элиминаторов

```
\func plus-commutes-zero (x : Nat) : x Nat.+ 0 = 0 Nat.+ x \elim x
  | 0 => idp                                      -- P 0
  | suc x => pmap suc (plus-commutes-zero x)    -- P x -> P (suc x)

-- Но то же можно сделать с помощью Nat-elim:

\func Nat-elim (P : Nat -> \Type)
               (z : P zero)
               (s : \Pi (n : Nat) -> P n -> P (suc n))
               (x : Nat) : P x \elim x
  | zero => z
  | suc n => s n (Nat-elim P z s n)

\func plus-commutes-zero' (x : Nat) : x Nat.+ 0 = 0 Nat.+ x =>
  Nat-elim (\lam x => x Nat.+ 0 = 0 Nat.+ x)
           (idp)
           (\lam x prev => pmap suc prev) x
```

Свойства равенства

Теорема

$a = a$. Если $a = b$, то $b = a$. Если $a = b$ и $b = c$, то $a = c$.

Доказательство.

Положим $f(x) := a$.
Положим $g(x) := f(1 - x)$. Тогда $f(0) = b$, $f(1) = a$, композиция непрерывных — непрерывна.
Пусть $f_1 : a \rightsquigarrow b$ и $f_2 : b \rightsquigarrow c$. Тогда

$$g(x) := \begin{cases} f_1(2 \cdot x), & x < 0.5 \\ f_2(2 \cdot x - 1), & x \geqslant 0.5 \end{cases}$$

$\square$

Равенство в Аренде

Компьютер дискретен, потому мы имитируем топологическую конструкцию.

- Интервал `I` := [left..right] — есть доступ только к граничным точкам, внутренность недоступна.
- Путь — индуктивный тип:

  ```
  \data Path (A : I -> \Type) (a : A left) (a' : A right)
    | path (\Pi (i : I) -> A i)
  ```

  path f : Path A a a' означает, что $f : a \rightsquigarrow a'$. Заметим, что компилятор дополнительно проверяет f left $\twoheadrightarrow_\beta a$ и f right $\twoheadrightarrow_\beta a'$ — неуказанная в определении встроенная семантика для `path`.
- Равенство $a = a'$ (примем $a, a' : A$) — сокращение:

  ```
  \func \infix 1 = {A : \Type} (a a' : A) => Path (\lam _ => A) a a'
  ```

  path $f : a = a'$ означает, что path f : Path A a a', то есть найдётся $f : a \rightsquigarrow a'$.

Как работать с I

```
\func \infix 1 = {A : \Type} (a a' : A) => Path (\lam _ => A) a a'
```

Докажем что-нибудь про равенство.

- Рефлексивность:
  ```
  \func eq-reflexive {A : \Type} {a : A} : a = a => path (\lam _ => a)
  ```
- Для транзитивности нужна середина интервала. Для этого воспользуемся элиминатором для I, также системное определение:

  ```
  \func coe (P : I -> \Type)
            (a : P left)
            (i : I) : P i \elim i
    | left => a
  ```

  «Если некоторый предикат P, индексированный интервалом, выполнен на левом крае интервала, то он выполнен в любой точке интервала»

Транзитивность равенства

```
\func eq-transitive {A : \Type} {a a' a'' : A}
                    (f : a = a')
                    (g : a' = a'') : a = a'' =>
  coe (\lam p => a = g @ p) f right
```

Проиллюстрируем картинкой:

$$
\begin{tikzcd}
a \arrow[r, "f : a = a'"] \arrow[dr] \arrow[ddr, "a = a''"'] & a' \arrow[d] & g@left \\
 & & g@p \\
 & a'' & g@right
\end{tikzcd}
$$

Здесь $p \in \mathtt{I}$, и $g@p$ — некоторая точка по пути g из a' в a''.

Симметричность

Осталось третье свойство: $a = a' \to a' = a$.

```
\func eq-symmetric {A : \Type} {a a' : A} (p : a = a') : a' = a \elim p
  | idp => idp
```

Сопоставление с образцом для равенства возможно — его обитатель есть `idp`, специальный конструктор. Также встроенное в язык поведение.

Стандартные функции работы с равенством

- Если $a = a'$ и $B(a)$ истинен, то $B(a')$ истинен.

```
\func transport {A : \Type} (B : A -> \Type) {a a' : A}
                (p : a = a') (b : B a) : B a'
     => coe (\lam i => B (p @ i)) b right
```

По сути тот же coe, но индексирует предикат вдоль пути вместо интервала:

```
\func eq-transitive {A : \Type} {a a' a'' : A}
                    (f : a = a')
                    (g : a' = a'') : a = a'' =>
  transport (\lam x => a = x) f right
```

- Если $a = a'$, то $f(a) = f(a')$:

```
\func pmap {A B : \Type} (f : A -> B) {a a' : A} (p : a = a') : f a = f a'
     => path (\lam i => f (p @ i))
```

Функциональная экстенциональность

Всюду совпадающие функции равны.

```
\func funExt {A : \Type} (B : A -> \Type) {f g : \Pi (a : A) -> B a}
    (p : \Pi (a : A) -> f a = g a) : f = g
    => path (\lam i => \lam a => p a @ i)
```

Доказать неравенство

- Ложь — это `Empty` (тип без значений).

```
\data Empty
```

- $0 \neq 1$ — это $0 = 1 \to \bot$.
- Построим функцию `T : Nat -> \Type`, которая для 0 возвращает обитаемый тип, а для других чисел — необитаемый.

```
\func T (x : Nat) : \Type \elim x
| Z => \Sigma
| _ => Empty
```

- T — это предикат, раз можем поставить вопрос, истинен (обитаем) ли $T\ 0$. Поэтому он сохраняет истинность для равных объектов (принцип Лейбница).
- Заметим, что `T 0` истинен (обитаем), так как `() : \Sigma`. Значит, мы найдём значение и для `T 1` (для `Empty`), при условии, что $0 = 1$.

```
zero-ne-1 (p : 0 = 1) : Empty => transport T p ()
```

$$\frac{A : \delta_1 \qquad B : \delta_2}{\Pi x^{A} . B}$$

A — тип.

B — тип.

$A : *$ $\qquad$ $B : *$

$\Pi x^{A} . B$. — пример!

$\Pi x^{\alpha} . \beta$ не зав. x $\qquad$ $\Pi x^{int} . char \equiv int \to char$

Напр.

$int[25]$, $\qquad$ $int[\,] : \cancel{int \to *}$ $\qquad$ возвр. тип.

$\Pi x^{int} . int[x] : \square$

$$\cancel{\frac{A : * \qquad B : \square}{\Pi x^{A} . B}}$$

3 пар.

map: $\underbrace{\forall L}$ → [L] → [L]

list. $\forall L$ → (тип списка L)

[1,2,3] : [int]*

$$\frac{\Gamma \vdash A : \sigma_1 \quad (*:\square) \qquad \Gamma, x : A \vdash B : \sigma_2^{*}}{\Gamma \vdash \Pi x^{A}.B : \sigma_2^{*}}$$

(2)

$$\frac{\cancel{*}:\Box \qquad \alpha:*}{\Pi\alpha^{*}.\alpha:*} \qquad \frac{A:B}{}$$

Каким образом:

$\Box, \Box$.

$\cancel{f:t}$ $\qquad$ $A:B$,

конст.

$\Lambda\alpha.\lambda x^{\alpha}.[\alpha]:\Pi\alpha^{*}.\alpha\ list$ — ф-я, кот. по типу α и зн. x строит $[x]$

vec!(x) $\qquad$ $\vdash \alpha list:*$ $\qquad$ $*:\Box$

ф-ю: $\Lambda\alpha.\alpha\ list:\Pi\alpha^{*}.*$